

REST API Specification Quality Analyzer

An Automated Implementation of Research-Based REST API Design Practices

Faraz Ahmed

Department of Computer Science, University of Sindh, Jamshoro, Pakistan
faraztech10@gmail.com

Abstract. This project presents an automated tool that checks whether a REST API follows recommended design practices. The tool reads an OpenAPI specification, applies rules derived from the catalog of 73 REST API design practices proposed by Prof. Fábio Petrillo and his colleagues, and produces a report with an overall score and a clear explanation for every rule. The system is built as two Spring Boot services. The analyzer parses the specification and runs 21 rules; the report service stores the results and returns them on request. The project demonstrates how a published research catalog can be turned into a practical, automated evaluation tool.

1 Introduction

REST APIs are everywhere, yet decisions about their design are usually made quickly and reviewed informally. A developer must choose resource names, HTTP methods, error responses, and pagination schemes, and these choices affect how easily other people can understand and use the API. Manual review depends on experience and tends to be slow and inconsistent.

This project was developed in line with the research of Prof. Fábio Petrillo, whose empirical study catalogued 73 best practices for REST API design, covering areas such as URI naming, HTTP method semantics, response codes, and API documentation. The aim of the project is to convert a subset of that catalog into executable rules. An API specification is checked mechanically, and every rule reports whether it passed, along with a message and a suggestion for improvement. The result is a review that is fast, consistent, and easy to justify, because each finding can be traced back to a published practice.

2 Objectives

The project has four goals:

1. accept an OpenAPI version 3 specification as input,
2. apply research-based design rules to every endpoint,
3. compute an overall quality score,
4. store the results and return them through simple HTTP requests.

3 Design and Implementation

The analysis follows a simple flow: parse, evaluate, score, store.

Parsing. The input is a JSON document in the OpenAPI format. The Swagger parser library reads it and converts it into a structured Java model containing the API version, server URLs, security schemes, and a list of endpoints with their parameters, request bodies, and responses. Working with a structured model keeps the rule code simple and readable.

Rules. Every rule is an independent class implementing one interface, so the engine treats them uniformly. Each rule receives an endpoint and the specification and returns a result with a status, a message, and a recommendation. Most rules apply per endpoint. A few, such as HTTPS server URLs and the presence of a security scheme, check the specification as a whole. Each rule references the practice identifier from Prof. Petrillo's catalog, which makes the connection between the tool and the research transparent.

A few example rules are shown below.

Rule	Practice area	Example check
Resource-oriented URI	URI naming	Rejects action verbs such as get or create in a path
Plural resource names	URI naming	Requires plural nouns for collection resources
HTTP method semantics	HTTP methods	Rejects a request body on GET, HEAD, or DELETE
POST returns 201	Response codes	Requires 201 Created on POST operations
HTTPS server URL	Operation	Requires all server URLs to use HTTPS

Scoring. Results are counted as passed or failed. The score is the percentage of passed rules among the evaluated rules, rounded to two decimal places. Results that are neither passed nor failed are excluded, since they carry no judgment. The score is deliberately simple: it is transparent and easy to explain, although it treats all rules as equally important.

Reporting. After evaluation, the analyzer sends the results to the report service over HTTP. The service saves the summary and each rule result in a database. Users can retrieve a full report, request only the failed rules, or list all previously analyzed APIs.

Architecture. The two services are separated on purpose. The analyzer handles parsing and evaluation; the report service handles storage and retrieval. Both follow a layered structure: a controller receives the request and delegates to a service interface, the service implementation contains the logic, and transformer classes convert data between requests, entities, and responses. This keeps each layer focused on one responsibility and makes the code easy to test and extend.

4 Testing

The project treats testing as part of its core work. The analyzer has around one hundred unit tests. Every rule has its own set of tests for passing and failing cases, including edge cases such as empty paths and missing fields. The report service runs integration tests that save an analysis into the database and read it back. A dedicated test also verifies that every rule maps to a valid practice identifier, since this mapping is what connects the tool to the research. All tests pass.

5 Limitations and Future Work

The tool is useful, but its limits are worth stating. First, it implements 21 of the 73 practices; the remaining ones are harder to check automatically. Second, several checks are heuristic. For example, the rule that rejects action verbs in URIs compares path segments against a fixed word list, so it catches clear cases such as /getUsers but cannot judge whether a name is meaningful for a specific resource. Third, the score treats every rule equally and says nothing about the importance of individual findings, so the score is always presented together with the per-rule results. Fourth, the tool inspects the specification document, not the running system, so it cannot verify actual server behavior.

The most immediate next step is extending the rule set. Other directions include weighting rules by severity, running the analysis in a CI pipeline, and comparing the tool's recommendations against real fixes made by developers.

6 Conclusion

This project shows that a published catalog of REST API design practices can be implemented as a working analysis tool. The system reads an OpenAPI specification, applies rules derived from the research of Prof. Fábio Petrillo, computes a transparent score, and stores the results for later review. The design keeps analysis and storage separate, and the rules are small and independently testable. The main contribution is a review process that is

automated, consistent, and explainable each finding can be traced to a published practice, which is exactly what a developer needs when deciding what to change.

References

- [1] Petrillo, F., Merle, P., Moha, N., and Guéhéneuc, Y. G. "[Are REST APIs for Cloud Computing Well-Designed? An Exploratory Study](#)." Proceedings of the International Conference on Service-Oriented Computing (ICSOC), 2016, pp. 157–170. Springer.
- [2] OpenAPI Specification, Version 3.0.3. OpenAPI Initiative. <https://spec.openapis.org/oas/v3.0.3>
- [3] Spring Boot Reference Documentation. <https://docs.spring.io/spring-boot/>
- [4] Swagger Parser. <https://github.com/swagger-api/swagger-parser>